SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis | 979-8-3503-5554-3/24/$31.00 ©2024 IEEE | DOI: 10.1109/SCW63240.2024.00208

Top-Down Microarchitecture Analysis Approximation Based on Performance Counter Architecture for SiFive RISC-V Processors

Chan-Yu Mou
Department of Computer Science, National Tsing Hua University
Software Performance Analysis, SiFive, Inc., Hsinchu, Taiwan
william.mou@lsalab.cs.nthu.edu.tw

Chao-Chieh Hsiao
Software Performance Analysis, SiFive, Inc., Hsinchu, Taiwan
adam.hsiao@sifive.com

Jerry Chou
Department of Computer Science, National Tsing Hua University
Hsinchu, Taiwan
jchou@lsalab.cs.nthu.edu.tw

***Abstract*—Modern Out-of-Order RISC-V CPUs have complex mechanisms, making microarchitecture-level performance analysis challenging. Despite increasing Performance Monitoring Units (PMUs), interpreting this data requires deep architectural knowledge. This paper introduces a Top-down Microarchitecture Analysis (TMA) approximation to analyze application performance on RISC-V CPUs. TMA classifies performance issues into four categories by calculating metrics that reflect their proportions using predefined formulas and PMU events. We present the results of applying this method to analyze SPEC CPU2006 benchmarks on a SiFive RISC-V processor. This work is an initial step in analyzing RISC-V CPUs using TMA Level 1. The contributions of this research are threefold: (1) designing and implementing TMA for a RISC-V CPU with clear metric definitions; (2) proposing test cases and methods to verify TMA metrics and PMU implementation; (3) enabling software developers to profile workloads without requiring extensive microarchitecture knowledge.**

***Index Terms*—CPU Microarchitecture, Performance analysis, RISC-V, Top-Down Analysis**

I. Introduction

RISC-V is an open Instruction Set Architecture (ISA) designed to be simple, flexible, and scalable, allowing anyone to use and modify it freely. Its openness and modular design have made RISC-V the preferred choice for hardware designers and developers. As diverse RISC-V processors are increasingly deployed in various devices, from microcontrollers to high-performance servers, there is a growing focus on their performance. Modern RISC-V out-of-order CPUs have Performance Monitoring Units (PMUs) to monitor various CPU events, but identifying actual performance bottlenecks within this vast data is challenging because performance counters have historically been constructed in a bottom-up fashion, as mentioned in the work of Stijn Eyerman and Lieven Eeckhout [1]. Therefore, there is a pressing need for a profiling method that analyzes application granularity to identify microarchitectural bottlenecks in out-of-order (OOO) RISC-V processors. This research is motivated by the necessity to develop such a profiling method to enhance the efficiency of RISC-V processors in diverse application domains.

This paper introduces a Top-Down Microarchitecture Analysis (TMA) method for RISC-V CPUs. The method creates an analysis hierarchy using metrics calculated from PMU counter values. It guides users through a hierarchical, level-by-level investigation of the bottleneck areas, providing a comprehensive performance overview from a top-down perspective. As demonstrated by Intel's paper [2], TMA has proven valuable for performance profiling, mainly through tools like VTune [3] and Perf [4]. These profiling tools offer three primary benefits: (1) a counter-based solution that enables statistical analysis for a better understanding of performance behaviors; (2) the use of preset metrics, allowing developers to optimize microarchitecture performance without requiring an in-depth understanding of complex microarchitecture details; and (3) high scalability and a unified interface, making the methodology applicable across various out-of-order processors. This enables developers to use similar analysis strategies and tools (e.g., Perf in Linux) across different architectures, such as AMD and Intel.

There are two main challenges to implementing the existing TMA solutions on RISC-V processors: (1) Directly referencing Intel and AMD's classifications is not feasible because some x86 terminology and microarchitecture components differ. For instance, Intel's Instruction Decode Queue (IDQ) is called the Instruction Queue in RISC-V, and the Register Alias Table (RAT) is known as the Map Table. Understanding the differences in microarchitecture and component functions between the two architectures is essential for designing TMA metrics aligned with x86, calculated using the appropriate PMU counters. (2) The differences in PMU design among processors affect the precision of the metrics and require approximation for use in RISC-V. For example, Intel's `IDQ_UOPS_NOT_DELIVERED` counter accurately measures the Frontend Bound metric, counting the number of uOps(micro-Operation) not delivered by the Frontend when the Backend is not stalled. But in RISC-V, the most accurate equivalent is the `decoded_less_than_maximum_operations` counter, which counts the number of cycles the decode stage is not

979-8-3503-5554-3/24/$31.00 ©2024 IEEE
DOI 10.1109/SCW63240.2024.00208

Authorized licensed use limited to: ShanghaiTech University. Downloaded on August 31,2026 at 04:37:43 UTC from IEEE Xplore. Restrictions apply.

fully utilized. Therefore, implementing Top-Down Microarchitecture Analysis (TMA) on RISC-V CPUs requires designing a Top-Down Analysis Hierarchy tailored to the RISC-V architecture and existing PMU design.

This paper proposes four categories in Level 1 of TMA: Retiring, Bad Speculation, Frontend Bound, and Backend Bound, demonstrating how to classify uOps into these categories through a detailed breakdown of the RISC-V microarchitecture. We design metric formulas for each category using the PMU of the SiFive RISC-V processor as an example, given its role as the inventor and primary promoter of RISC-V. Finally, we showcase the effectiveness of our method using the well-known benchmark application, SPEC CPU2006, to analyze program behavior on RISC-V and compare the result with other processors.

This paper also discusses how to verify the correctness of the metric design and PMU implementation by checking the profiling results against the following test cases to ensure they meet expectations. We selected two case studies for verification, each with two versions: before and after optimization. The first case is expected to show a reduction in Bad Speculation, while the second case is anticipated to reveal the Backend Bound impact caused by memory access.

In this paper, we present three key contributions:

1) This is the first attempt to model the TMA method on a RISC-V CPU, providing explicit definitions of the top four metric formulas for RISC-V.
2) We propose a set of test cases, methods, and results to verify that the TMA metrics design and PMU implementation are reasonable.
3) This methodology offers a straightforward, counter-based solution and aids software developers in profiling workloads without deep microarchitecture knowledge.

The rest of this paper is organized as follows: Section II provides background information and presents the main assumptions of TMA method. Section III details the proposed SiFive TMA method and defines the metrics formula of Level 1. Section IV showcases our method's effectiveness on SPEC CPU2006 workloads by analyzing Level 1 distribution across different microarchitectures. It also validates the results through two case studies. Section V discusses related work. Finally, Section VI concludes the paper and outlines future work.

II. BACKGROUND

Modern microprocessors are equipped with various mechanisms to enhance execution efficiency. However, due to their highly interconnected design, even minor issues can significantly impact their performance potential. Identifying the true performance bottlenecks among numerous minor issues remains a significant challenge in the field. Traditional CPU performance monitoring methods, often referred to as "naïve approaches," are inadequate for modern out-of-order CPUs for several reasons, as discussed in Intel's paper [2] on naïve approach design. The Top-Down Microarchitecture Analysis (TMA) method, derived from the Top-Down Analysis approach, aims to pinpoint the actual performance-limiting factors among the many hidden issues. In this Section, we will introduce two key concepts: Hierarchy and Slots.

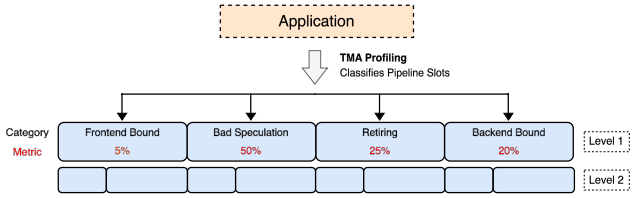


Fig. 1: Top-Down Microarchitecture Analysis (TMA) hierarchy tree structure, with performance metrics indicated at various levels.

A. TMA hierarchy concept

The TMA method uses a hierarchical tree structure, as shown in Figure 1, with levels labeled on the right side of the diagram. Each level is divided into multiple categories, and the corresponding metrics (highlighted in red) quantify the proportion of performance impact for each category. For example, Level 1 is divided into four categories, with the metric for "Bad Speculation" showing 50%, indicating that 50% of uOPs are stalled in this category. These values are calculated using predefined formulas based on PMU (Performance Monitoring Unit) event counts.

When using TMA's Hierarchical analysis, we need to follow these two rules:

- Contextual Node Validation: Only consider the values of inner nodes if the entire path from the root is flagged.
- Sibling Nodes Comparison: Weights are only comparable between sibling nodes at the same pipeline stage.

By organizing performance issues within this hierarchical framework, TMA provides a clear and systematic approach to identifying and addressing inefficiencies in a processor's microarchitecture.

B. TMA Slot concept

TMA focuses on performance analysis at the instructions level, leveraging the concept of pipeline Slots from Intel's paper [2]. Slots are defined as the product of the pipeline width and the number of cycles. This term represents the maximum uOps the CPU can deliver to the Backend over a given period. The Slots concept is used to observe uOps allocation, with each slot representing whether an uOP is delivered from the Frontend (in-order execution) to the Backend (out-of-order execution).

For example, assuming a CPU with a pipeline width of three, we can represent this using the slot concept shown in Figure 2. Gray represents unused resources, and green represents used ones. In an ideal scenario, the Frontend can send three uOps to the Backend each cycle, and the Backend can also receive three uOps per cycle. This would result in a situation where three green slots are filled, as seen in the first cycle in Figure 2. If the Frontend can only send one uOp or

Authorized licensed use limited to: ShanghaiTech University. Downloaded on August 31,2026 at 04:37:43 UTC from IEEE Xplore. Restrictions apply.

the Backend cannot receive more than one uOp, it would lead to situations where pipeline slots are not filled with uOps. As shown in the third cycle in Figure 2, the gray areas represent pipeline slots that are not filled with uOps, for example, due to stalled or lack of resources in subsequent stages.

In summary, as shown in Figure 2, the green slots indicate that seven uOps entered the Backend over these three cycles. Two slots are without uOps, meaning the other two ALUs were idle, waiting for the stalled slot.

In Section III, we'll explain how to categorize these slots into four Level 1 categories and outline how to calculate the metrics for each. This will help identify and understand key performance bottlenecks for RISC-V CPU.

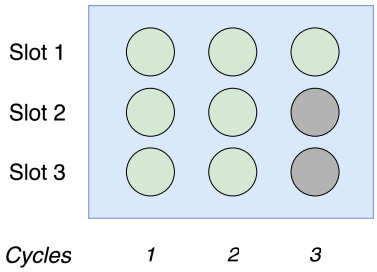


Fig. 2: Pipeline slot concept in a CPU with a width of three uOps per cycle

III. Proposed SiFive TMA Level 1 Metrics

In this work, we aim to provide Level 1 metrics designed for the SiFive Processor using the Top-Down Microarchitecture Analysis (TMA) method. The method outlined here can be used as a guide for other RISC-V CPU designers to develop their own TMA Level 1 metrics tailored to their architectures.

To minimize confusion due to terminology differences, we describe the distinctions between x86 and SiFive terms in Appendix 1. Section III-A defines the top-level breakdown flowchart for RISC-V. Sections III-B to III-E detail the PMUs required for each metric in RISC-V and their significance. Finally, section III-F summarizes the formulas for the metrics.

A. Breakdown

Depending on the microarchitecture design, metrics suitable for one architecture often do not apply to another. This paper uses x86 metrics as a reference and provides revised metrics tailored to the SiFive core design.

In this paper, the breakdown of the four primary flow categories is shown in Figure 3: Frontend Bound, Backend Bound, Bad Speculation, and Retiring. Every instruction executed on the processor is classified into one of these categories. The natural division between the Frontend and Backend of the processor makes them the first two metrics in the hierarchy. Depending on the execution status, a micro-operation (uOP) is classified into one of four categories. "Retiring" indicates that a uOP has been successfully dispatched and retired. "Bad Speculation" occurs when a uOP is dispatched but not retired. "Backend bound" means a uOP failed to dispatch due to a backend stall, while "Frontend bound" refers to a uOP that failed to dispatch due to an issue unrelated to a backend stall.

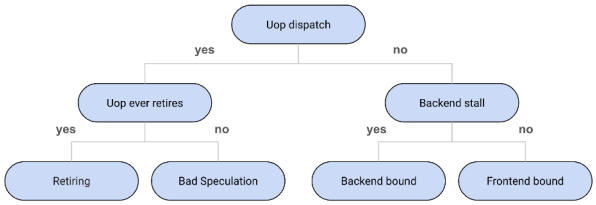


Fig. 3: Breakdown of four primary categories in TMA: Frontend Bound, Backend Bound, Bad Speculation, and Retiring.

Frontend denotes the first part of the processor core responsible for fetching instructions that are executed later on by the Backend part. Within the Frontend, a branch predictor predicts the next address to fetch; cache lines are fetched from the memory subsystem; the decoder parses instructions into uOps; and at the beginning of the Backend, the rename part holds and updates mappings from ISA registers to physical registers. Ideally, the Frontend can fill the pipeline width with uOps to the Backend each cycle, such as filling three uOps with 3-wide pipes in Figure 4. Any deviation from this saturation due to the Frontend itself would be categorized as Frontend bound. For example, there are stalls due to instruction cache misses.

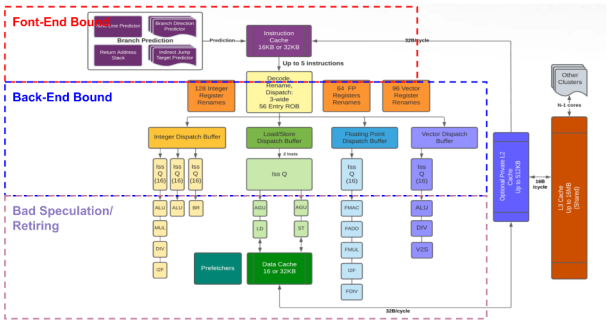


Fig. 4: SiFive P470 detailed pipeline overview and bounding of TMA categories

B. Frontend Bound Category

Intel's work calculates this metric using a performance counter, `IDQ_UOPS_NOT_DELIVERED.CORE`, which counts the number of uOps not delivered from IDQ to RAT while the RAT is not stalled, as illustrated in Figure 11 at the boundary between yellow and green. In AMD's work, the `DECODER_EMPTY` counter counts the number of cycles where the decoder has nothing to dispatch. This leads to three inaccuracies:

1) Problem 1. The decoder being empty is not equivalent to dispatching nothing because the instructions in the

Authorized licensed use limited to: ShanghaiTech University. Downloaded on August 31,2026 at 04:37:43 UTC from IEEE Xplore. Restrictions apply.

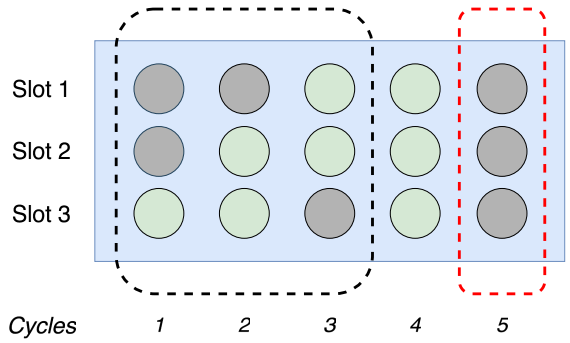


Fig. 5: Illustration of partial decoding and empty slots in a 3-wide pipeline

Instruction Dispatch Queue (IDQ) come from multiple sources.
2) Problem 2. This counter will also include Backend stall situations.
3) Problem 3. Empty slots during partial decoding are not recorded, for example, when the decoder only fills 1 or 2 uOps in 3-wide pipes. Show in the black block in Figure 5.

In SiFive core, we selected the `decoded_less_than_maximum_operations` counter to approximate the Frontend Bound metric. This counter counts cycles in which the Instruction Fetch Unit delivered less than the maximum possible number of instructions to decode, as shown in the black block in Figure 5. Therefore, the counter records events that involve only partial decoding. Let's consider each of the problems mentioned in the last paragraph one by one:

1) Problem 1. In the SiFive Core, the decoder directly passes instructions to the rename stage after dequeuing from the fetch queue, as shown in Figure 4, therefore there will be no inaccuracies
2) Problem 2. This counter will not increment if the decode stage is stalled, excluding the cycles caused by Backend stalls.
3) Problem 3. The counter can only count the cycles of partial decode. This means we cannot determine the exact number of empty slots in each cycle, as shown in the red block in Figure 5. we only know how many cycles involve partial decode.

The first and second issues do not result in inaccuracies on SiFive's PMU. We further investigate how to reduce the inaccuracies from the third problem mentioned above. For the number of cycles where the decoder is empty, we can include all PMU events that cause the decoder to empty to achieve an approximation. For example, mispredictions invariably lead to an empty decoder. For partial decode, we multiply the cycles by a constant number to represent the empty slots. This will be introduced in section III-F.

C. Bad Speculation Category

Bad Speculation measures the slots lost due to incorrect speculations, primarily from mispredicted branch instructions. In Intel's work, this metric is defined as `(UOPS_ISSUED.ANY - UOPS_RETIRED.RETIRE_SLOTS) + (Pipeline_Width × INT_MISC.RECOVERY_CYCLES)`. The left side of the formula represents uOps delivered to the Backend but not retired, while the right side indicates the number of slots waiting for the microarchitectural state in the Resource Allocation Table (RAT) to be recovered after a nuke due to various cases. Combined, these are categorized as Bad Speculation.

For the left side, SiFive processors require a performance counter to count `dispatch`, referred to `issue` in Intel terminology as shown in Table V. Therefore, this paper proposes an approach that attempts to list all events causing Bad Speculation explicitly. Similar to AMD's approach, RISC-V does not measure how many cycles are affected by Bad Speculation, only the number of occurrences. Hence, we approximate by adding a misprediction penalty to represent the average number of false path instructions per event. The specific value of this cost depends on the processor. In the experiments presented in this paper, a value of 9 cycles multiplied by a pipeline width of 3, resulting in 27 slots, was used. However, this limitation means that the branch misprediction penalty cannot accurately reflect the number of flushed instructions caused by a branch misprediction. For the right side of the formula, we use `instruction_decode_stall_from_recover` to approximate the recovery cycles due to misspeculation, analogous to `INT_MISC.RECOVERY_CYCLES` in Intel's metric.

D. Retiring Category

This category represents the resources utilized by helpful work, i.e., dispatched instructions that eventually get retired. Ideally, all instructions in the pipeline would be attributed to the Retiring category. A Retiring value of 100% would indicate that the maximum Pipeline Width throughput was achieved. Maximizing Retiring typically increases the Instructions-per-Cycle (IPC) metric.

The "instructions" counter represents the Retiring category, which counts retired instructions. This approach is similar to Intel's, where the `UOPS_RETIRED.RETIRE_SLOTS` is used.

It is important to note that a high Retiring value does not necessarily imply no room for further performance improvement. For example, RVV instructions can reduce the total number of instructions, improving performance.

E. Backend Bound Category

The Backend represents the second part of the processor, which receives instructions from the Frontend and is responsible for monitoring when microoperation data operands are available and executing the microoperation in an available execution unit. Backend Bound reflects the situations when no microoperations are delivered to the issue queues as the Backend runs out of some resources, e.g., load buffers.

Authorized licensed use limited to: ShanghaiTech University. Downloaded on August 31,2026 at 04:37:43 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Definitions of all SiFive RISC-V Top-Down Characterization metrics

Metric Name	Linux Perf Metrics Formula
Frontend Bound	(decoded_less_than_maximum_operations * *$PARTIAL_SLOT_FACTOR* + (branch_direction_misprediction + ijtp_misprediction + ras_mispredicted_target) * *$FETCH_WAIT_CYCLE* * *$PIPELINE_WIDTH*) / (cycles * *$PIPELINE_WIDTH*)
Bad Speculation	((branch_direction_misprediction + ijtp_misprediction + ras_mispredicted_target)* *$BPM_COST* * *$PIPELINE_WIDTH* + instruction_decode_stall_from_rhf_recover) * *$PIPELINE_WIDTH)* / (cycles * *$PIPELINE_WIDTH*)
Backend Bound	1-(Frontend Bound + Bad Speculation + Retiring)
Retiring	Instructions / (cycles * *$PIPELINE_WIDTH*)

TABLE II: Constant values for SiFive P470 core

Const Variable	Description	Value
BPM_COST	The number of cycles of Branch misprediction Cost	9
PARTIAL_SLOT_FACTOR	We multiply the cycles by this number for partial decode to represent the partial slots.	2
FETCH_WAIT_CYCLE	The number of cycles of instruction queue waits for fetch after a flush.	2
PIPELINE_WIDTH	The maximum number of instructions that the CPU can deliver to the Backend	3

The Backend Bound category is calculated as 100% minus the sum of all three other metrics at the top of the hierarchy. Since a microoperation may get classified only into one of the four top categories (Frontend Bound, Bad Speculation, Retiring, or Backend Bound), calculating Backend Bound in this way gives correct results and is the simplest method.

F. Metrics Formulas

This section provides a summary of the specific performance counters used to derive the metrics in the SiFive RISC-V Top-Down Microarchitecture Analysis.

Table I presents the formulas of RISC-V TMA metrics based on the performance counters available in SiFive processors. The significance of these performance counters has already been discussed in previous sections. Formulas prefixed with a $ sign and indicated in *italicized* text denote auxiliary expressions. Table II provides the values of these auxiliary expressions for SiFive processors.

IV. Result

In this section, we present Top-Down Microarchitecture Analysis results for the SPEC CPU2006 INT benchmark that illustrate the practical applications of the presented method. To validate the effect of the TMA methodology in SiFive RISC-V architecture, two case studies were conducted: 1) Branch misprediction and 2) Matrix multiplication. These experiments also demonstrate how TMA can optimize code by addressing Bad Speculation and Backend Bound issues respectively. These sections include:

- An overview of the values of metrics for different integer benchmarks from the SPEC CPU2006 INT benchmark (Section IV-B)
- An analysis detailing the similarities and differences in distribution of SPEC CPU2006 between Intel and AMD processors (Section IV-B)
- An analysis of branch misprediction and its impact on performance (Section IV-C)
- A comparison of the performance metrics for matrix multiplication in both optimized and non-optimized code versions (Section IV-D).

TABLE III: System Parameters

Processor	SiFive Performance™ P470, 32.5MHz, L1 I-Cache 32KB, L1 D-Cache 32KB, non L2 Cache, L3 Cache 2MB
Memory	2GB
Kernel	Linux 6.7.9
Benchmark	SPEC 2006CPU v1.2 (not optimized version)
Compiler	SiFive internal clang (close to upstream clang 18)

A. Results for the SPEC CPU2006 INT benchmark

The values of the four top metrics for the SPEC CPU2006 benchmark are presented in Fig. 6. We use the non-optimized version to highlight the distribution graph of TMA. The integer benchmarks comprise the first 12 workloads (from *400.perlbench* to *483.xalancbmk*) while the floating point benchmarks have not yet been covered. In this figure, the large yellow proportion in *429.mcf* indicates that this workload is primarily Backend Bound. The large red proportion in *473.astar* indicates that performance is affected by Bad Speculation.

SPEC2006 optimizers can obtain the following results and recommendations based on the TMA analysis:

- Workloads with high Bad Speculation include *401.bzip2*, *445.gobmk*, *458.sjeng*, and *473.astar*. It is recommended that branch prediction be improved to enhance performance. A simple optimization case will be described in section IV-C.
- Workloads with high Backend Bound include *429.mcf*, *462.libquantum*, and *471.omnetpp*, which have a low IPC. This result aligns with the findings of the paper "Memory Characterization of Workloads Using Instrumentation-Driven Simulation," which identifies these three workloads as memory bound. A simple optimization case will be described in section IV-D.

B. Validate SPEC CPU2006 with Intel and AMD Processors

Figures 7 illustrate the Retiring, Backend Bound, Frontend Bound, Bad Speculation, and IPC metrics for the SPEC CPU2006 suite on Intel (Figures 7 (a)) and AMD (Figures 7 (b)) processors as referenced from the publication introducing the Intel Top-Down method [2] and an AMD paper [5]. A similar distribution of the IPC metric is easily visible across

Authorized licensed use limited to: ShanghaiTech University. Downloaded on August 31,2026 at 04:37:43 UTC from IEEE Xplore. Restrictions apply.

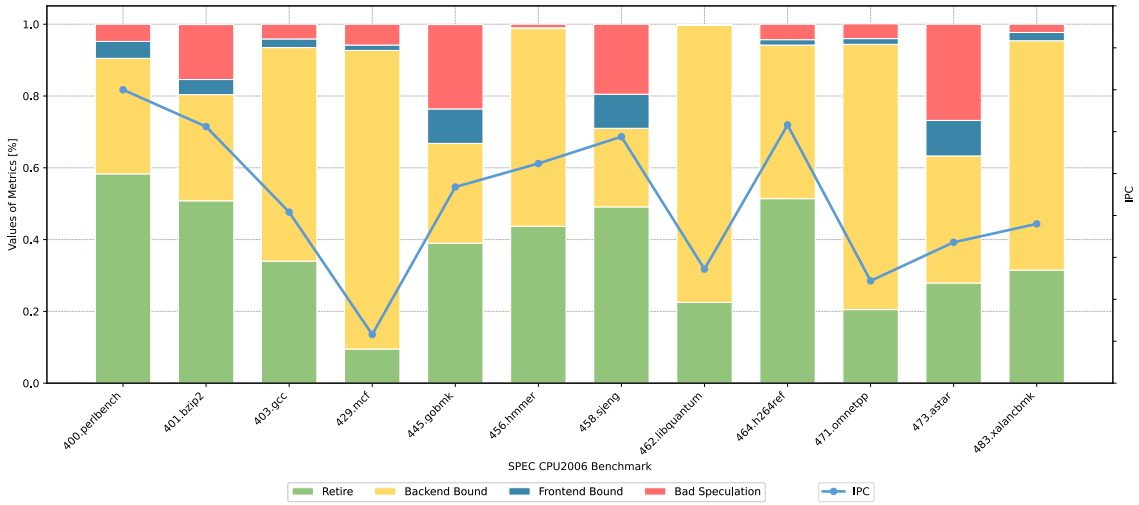
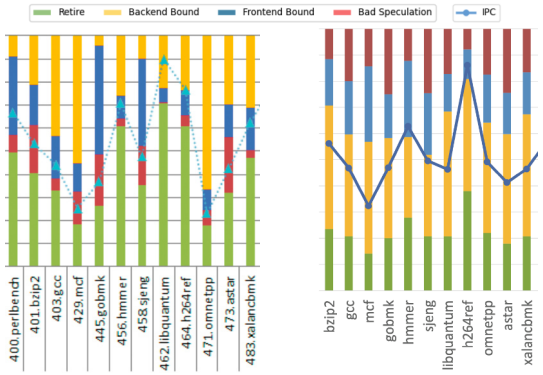


Fig. 6: Performance distribution for SPEC CPU2006 benchmarks, showcasing TMA metric results for the RISC-V SiFive processor

benchmarks, which is also reflected in the Retiring metric due to its correlation with IPC.



(a) Intel Processor (b) AMD Processor

Fig. 7: TMA Result for the SPEC CPU2006 on Intel and AMD Processor.

For example, the IPC values for the first four benchmarks decrease (note that the AMD paper does not include *400.perlbench*), then they increase up to *456.hmmer* with slight variations observed for *458.sjeng*. Notably, the values for *462.libquantum* differ significantly – it is quite low for AMD and SiFive processors, whereas it is the highest for Intel. We observed an inverse relationship between IPC and Backend Bound. The differences in Backend Bound may arise from variations in cache architecture design. Based on current P470 performance counters, it points out the Backend Bound is around cache components but doesn't distinguish the exact bottleneck for every case. Other analysis approaches, or newer counters might be useful for further investigation, as discussed in VI. Despite these variations, the lowest IPC values are consistent across all three processors. For the Intel processor, the lowest IPC is observed in the *471.omnetpp* and *429.mcf* benchmarks. For AMD and SiFive, *429.mcf* has the lowest IPC while *471.omnetpp* also shows relatively low IPC values.

C. Case Study 1: Branch Misprediction

The target program, as shown in Algorithm 1, aims to demonstrate the impact of branch mispredictions on performance using an example of conditional summation with an unsorted array. The following code snippet generates an array of random values and then sums the values that meet a specific condition, optionally `#ifdef ENABLE_SORT` sorting the array first.

```
const unsigned arraySize = 32768;
int data[arraySize];

for (unsigned c = 0; c < arraySize; ++c)
    data[c] = std::rand() % 256;

#ifdef ENABLE_SORT
std::sort(data, data + arraySize);
#endif

long long sum = 0;
for (unsigned i = 0; i < 1000; ++i) {
    for (unsigned c = 0; c < arraySize; ++c) {
        if (data[c] >= 128)
            sum += data[c];
    }
}
```

Algorithm 1: Code snippet for branch misprediction analysis

Figure 8a shows the memory layout when it is unsorted, where random values are generated. The program iterates through the loop, checking `(data[c] ≥ 128)`, which can

Authorized licensed use limited to: ShanghaiTech University. Downloaded on August 31,2026 at 04:37:43 UTC from IEEE Xplore. Restrictions apply.

result in multiple branch mispredictions, as indicated by the TRUE and FALSE evaluations in the diagram.

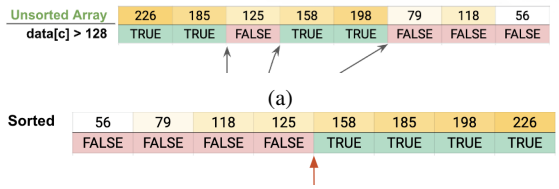
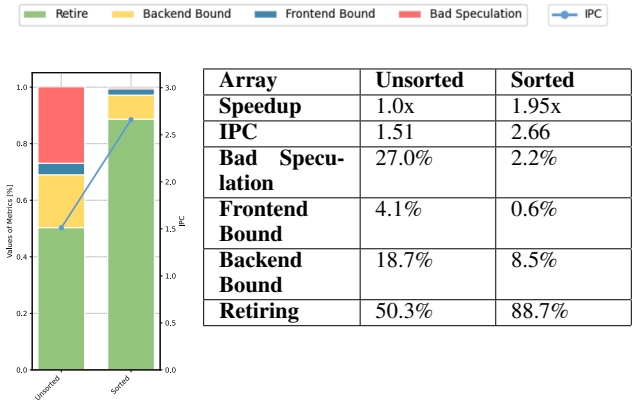


Fig. 8: Comparison of unsorted (a) and sorted (b) arrays.



Array	Unsorted	Sorted
Speedup	1.0x	1.95x
IPC	1.51	2.66
Bad Speculation	27.0%	2.2%
Frontend Bound	4.1%	0.6%
Backend Bound	18.7%	8.5%
Retiring	50.3%	88.7%

(a) (b)

Fig. 9: Branch Misprediction Performance Comparison

If we enable conditional compilation block `#ifdef ENABLE_SORT`, which sorts the array, we can reduce branch mispredictions because the values are now ordered. When the program checks if each value is greater than or equal to 128, the pattern becomes more predictable, leading to fewer mispredictions. Figure 8b clearly shows that after sorting, the branch is more predictable.

The key takeaway from these results is the significant reduction in Bad Speculation from 27.0% in the unsorted program to 2.2% in the sorted program, as shown in Figure 9 (a). This reduction highlights how sorting the array leads to more predictable branching, which minimizes branch mispredictions and boosts overall performance. Additionally, the execution time improves from 0.083644 seconds to 0.042969 seconds, demonstrating a performance gain of about 2x as shown in Figure 9 (b). The additional sorting overhead is small and can be ignored.

By utilizing the perf tool with SiFive TMA metrics, we can efficiently identify performance bottlenecks and make informed decisions to optimize our programs. In the next section, we will demonstrate how to use TMA to optimize the performance of a matrix multiplication program incrementally.

```
int **mmult(int rows, int cols, int **m1, int **m2,
    int **m3) {
    int i, j, k, val;
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) {
            for (k = 0; k < cols; k++) {
                m3[i][j] += m1[i][k] * m2[k][j];
            }
        }
    }
    return m3;
}
```

Algorithm 2: Original matrix multiplication code

```
int **mmult_order(int rows, int cols, int **m1, int
    **m2, int **m3) {
    int i, j, k, val;
    for (i = 0; i < rows; i++) {
        for (k = 0; k < cols; k++) {
            for (j = 0; j < cols; j++) {
                m3[i][j] += m1[i][k] * m2[k][j];
            }
        }
    }
    return m3;
}
```

Algorithm 3: Optimized matrix multiplication code with loop interchange

D. Case Study 2: Matrix multiplication

The target program aims to demonstrate the Backend bottleneck using a matrix multiplication example from the LLVM test suite. The

In the Algorithm 2 snippet, the `mmult` function calculates the product of matrices $\mathbf{m1}$ and $\mathbf{m2}$ using three nested loops and accumulates the result into matrix $\mathbf{m3}$ as $\mathbf{m3} \mathrel{+}= \mathbf{m1} \times \mathbf{m2}$

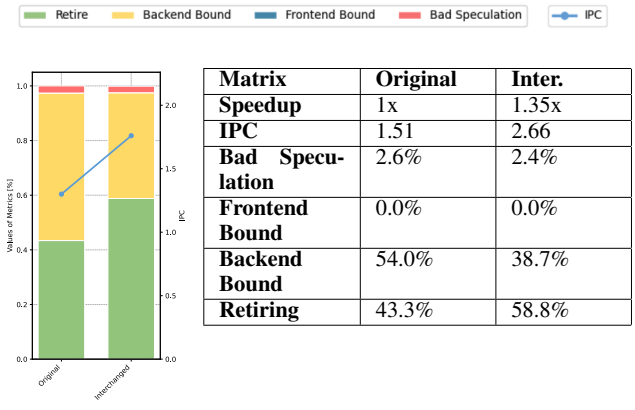
Algorithm 2 is known to have memory access performance issues, primarily due to non-sequential accesses to arrays. In this context, the problem arises because elements within a row of an array are arranged sequentially in memory. Specifically, the fastest changing array index, k, accesses a different row of m2 in each iteration. This access pattern leads to frequent cache misses, resulting in a high percentage of Backend Bound (54.0%), as shown in Figure 10 (a).

In Algorithm 3, after applying the loop-interchange refinement, the overall speedup is 1.35x (from 29,191,244,411 cycles to 25,893,595,525 cycles). Additionally, the Backend Bound is reduced from 54.0% to 38.7%, as shown in the yellow proportion in Figure 10 (b). This optimization demonstrates the effectiveness of loop interchange in mitigating memory access issues and improving overall performance.

V. Related Work

This work refers to the Intel Top-Down Method initially presented in [2]. It is the first attempt to implement this method in RISC-V processors.

Authorized licensed use limited to: ShanghaiTech University. Downloaded on August 31,2026 at 04:37:43 UTC from IEEE Xplore. Restrictions apply.



Matrix	Original	Inter.
Speedup	1x	1.35x
IPC	1.51	2.66
Bad Speculation	2.6%	2.4%
Frontend Bound	0.0%	0.0%
Backend Bound	54.0%	38.7%
Retiring	43.3%	58.8%

(a) (b)

Fig. 10: Matrix Multiplication Performance Comparison

In the past, there were multiple approaches to performance evaluation, with some of the most well-known being simple methods such as those detailed in [6] and [7] While these methods provided accurate results for in-order CPUs, out-of-order processors introduced more complex mechanisms during execution, such as overlapping stalls, speculative execution, and varying penalties for different workloads, rendering these methods imprecise.

In some cases, such as those by [8] and [9], algorithms or simulations are used to predict processor performance by exploring design space through multiple simulations. While these methods provide performance estimates, they are often slow and lack detailed insights into specific execution internals, making it difficult to identify precise performance bottlenecks.

In other cases, like [10] and [11] propose methods for creating Cycles per Instruction (CPI) stacks for superscalar processors. These methods break down a program's execution time into various cycle components, showing how many cycles are lost due to different miss events, such as cache and TLB misses. They use interval analysis as the foundation of their models, noting that under ideal conditions, a superscalar processor can achieve an IPC rate close to its pipeline width. However, these approaches are based on analytical modeling and do not offer detailed insights into the internal workings of a specific architecture.

In addition to software solutions, papers like [12] aim at providing hardware support for instruction-level profiling on out-of-order processors. More recent work, such as [13], introduces Time-Proportional Event Analysis (TEA), which explains execution time by mapping commit stalls to the performance events that caused them—enabling the creation of time-proportional Per-Instruction Cycle Stacks (PICS). TEA offers analysis results that are close to a golden reference, but unlike TMA, it requires hardware design tailored to a specific architecture for implementation.

VI. Summary and Future Work

This paper presented Top-Down Microarchitecture Analysis Approximation based on SiFive RISC-V processors, a methodology designed to identify actual bottlenecks in out-of-order processors. Approximate metrics are proposed in this paper and imply several performance counters can be revised or added to calculate the precise counts for TMA analysis. The described metrics reflect the efficiency of various processor parts and are derived from values of available performance counters. This methodology can be applied to analyze the execution of both single and multithreaded applications across a wide range of workloads. It is helpful in various scenarios, from optimizing applications during the development phase to conducting detailed execution analyses and investigating processor efficiency.

The authors hope this method will incentivize SiFive and other RISC-V design manufacturers to equip their future processors with more performance counters, delivering more accurate metrics and providing deeper insights into execution internals. The presented categories might be further divided into subcategories, such as level 2 and level 3, in the future. For example, the Backend Bound at level 2 could include memory bound or core bound, while level 3 could cover the efficiency of different ALUs. This would offer even more detailed information about performance bottlenecks.

Acknowledgement

The authors extend their sincere thanks to Green Wan, Josh Smith, Nicolas Brunie, Eric Lin, Bruce Ableidinger, and the other SiFive reviewers for their invaluable comments and suggestions. These individuals' contributions, along with the collaborative environment they fostered, were instrumental in supporting the research presented in this paper.

References

[1] S. Eyerman, L. Eeckhout, T. Karkhanis, and J. E. Smith, "A performance counter architecture for computing accurate cpi components," in *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS XII. New York, NY, USA: Association for Computing Machinery, 2006, p. 175–184. [Online]. Available: https://doi.org/10.1145/1168857.1168880

[2] A. Yasin, "A top-down method for performance analysis and counters architecture," in *2014 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2014, pp. 35–44.

[3] Intel Corporation. (2024) Intel vtune profiler. Accessed: 2024-08-15. [Online]. Available: https://www.intel.com/content/www/us/en/developer/tools/oneapi/vtune-profiler.html#gs.i6xhgk

[4] A. C. De Melo, "The new linux'perf'tools," in *Slides from Linux Kongress*, vol. 18, 2010, pp. 1–42.

[5] M. Jarus and A. Oleksiak, "Top-down characterization approximation based on performance counters architecture for amd processors," *Simulation Modelling Practice and Theory*, vol. 68, pp. 146–162, 2016. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1569190X16302258

[6] A. Ailamaki, D. J. DeWitt, M. D. Hill, and D. A. Wood, "Dbmss on a modern processor: Where does time go?" in *Proceedings of the 25th International Conference on Very Large Data Bases*, ser. VLDB '99. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1999, p. 266–277.

[7] D. Levinthal. (2009) Performance analysis guide for intel® core™ i7 processor and intel® xeon™ 5500 processors. [Online]. Available: https://www.intel.com/content/dam/develop/external/us/en/documents/performance-analysis-guide-181827.pdf

Authorized licensed use limited to: ShanghaiTech University. Downloaded on August 31,2026 at 04:37:43 UTC from IEEE Xplore. Restrictions apply.

[8] J. Lee, H. Jang, and J. Kim, "Rpstacks: Fast and accurate processor design space exploration using representative stall-event stacks," in *Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-47. USA: IEEE Computer Society, 2014, p. 255–267. [Online]. Available: https://doi.org/10.1109/MICRO.2014.26

[9] D. Shen, Q. Luo, D. Poshyvanyk, and M. Grechanik, "Automating performance bottleneck detection using search-based application profiling," in *Proceedings of the 2015 International Symposium on Software Testing and Analysis*, ser. ISSTA 2015. New York, NY, USA: Association for Computing Machinery, 2015, p. 270–281. [Online]. Available: https://doi.org/10.1145/2771783.2771816

[10] S. Eyerman, L. Eeckhout, T. Karkhanis, and J. E. Smith, "A performance counter architecture for computing accurate cpi components," *SIGOPS Oper. Syst. Rev.*, vol. 40, no. 5, p. 175–184, oct 2006. [Online]. Available: https://doi.org/10.1145/1168917.1168880

[11] T. S. Karkhanis and J. E. Smith, "Automated design of application specific superscalar processors: an analytical approach," *SIGARCH Comput. Archit. News*, vol. 35, no. 2, p. 402–411, jun 2007. [Online]. Available: https://doi.org/10.1145/1273440.1250712

[12] J. Dean, J. Hicks, C. Waldspurger, W. Weihl, and G. Chrysos, "Profileme: hardware support for instruction-level profiling on out-of-order processors," in *Proceedings of 30th Annual International Symposium on Microarchitecture*, 1997, pp. 292–302.

[13] B. Gottschall, L. Eeckhout, and M. Jahre, "Tip: Time-proportional instruction profiling," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 15–27. [Online]. Available: https://doi.org/10.1145/3466752.3480058

[14] J. Doweck, W.-F. Kao, A. K.-y. Lu, J. Mandelblat, A. Rahatekar, L. Rappoport, E. Rotem, A. Yasin, and A. Yoaz, "Inside 6th-generation intel core: New microarchitecture code-named skylake," *IEEE Micro*, vol. 37, no. 2, pp. 52–62, 2017.

[15] I. SiFive, "Sifive performance series: P450 and p470 cores," 2024, accessed: 2024-08-15. [Online]. Available: https://www.sifive.com/cores/performance-p450-470

[16] C. Celio, D. A. Patterson, and K. Asanovic, "The berkeley out-of-order machine (boom): An industry-competitive, synthesizable, parameterized risc-v processor," *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2015-167*, 2015.

[17] Advanced Micro Devices, Inc., *Software Optimization Guide for the AMD Zen4 Microarchitecture*, 2023, section 1.3. [Online]. Available: https://www.amd.com/en/support/tech-docs

APPENDIX

Since x86-64 and RISC-V share many microarchitecture concepts, although they have different names, Table IV lists how terminology can be mapped between the various implementations as a reference for the reader. For the x86 architecture, the Skylake [14] microarchitecture is the reference. For the RISC-V architecture, the SiFive P470 [15] microarchitecture is the reference.

TABLE IV: Microarchitecture differences between x86-64 and SiFive

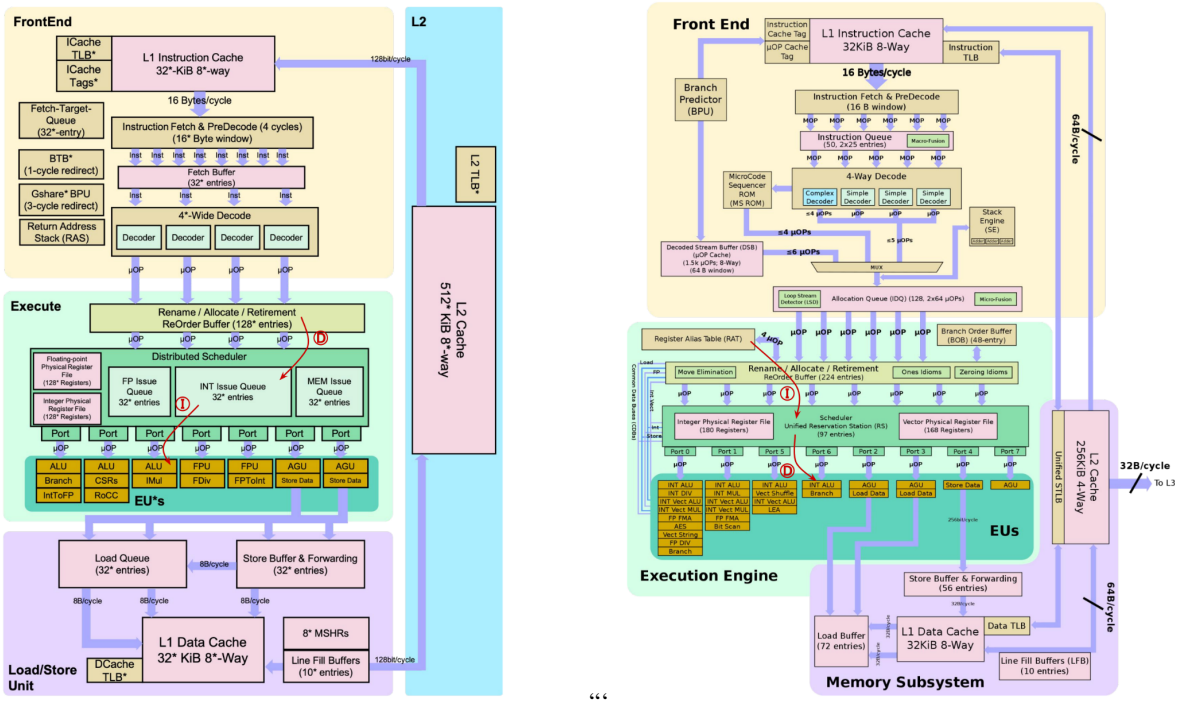
x86-64	**RISC-V (P470)**	**Description**
IDQ(Intel) MicroOP-Queue(AMD)	Instruction queue	The last queue before the "not yet renamed" instructions
RAT / Alloc	Map Table	Stores the mapping of architectural registers to physical registers
RS	Issue Queue	The pool of "not yet executed" instructions

Table V employs the BOOM [16] architecture, a general RISC-V diagram, to highlight CPU pipeline stage terminology commonly found in PMU descriptions. The terminology of RISC-V is more closely aligned with that of AMD. In this paper, the conventional terminology of RISC-V is used, except where original definitions from Intel or AMD are specifically referenced.

Authorized licensed use limited to: ShanghaiTech University. Downloaded on August 31,2026 at 04:37:43 UTC from IEEE Xplore. Restrictions apply.

TABLE V: Terminology mapping between RISC-V, Intel, and AMD

Terminology	**RISC-V** [16]	**Intel** [14]	**AMD** [17]
Dispatching (Label with Ⓓ and shown in Figure 11)	After renaming, the uOp is then dispatched or written into a set of Issue Queues.	Sending a uOp to an execution port from the reservation station.	Transferring macro ops from the processor's Front End to the out-of-order Backend.
Issuing (Label with Ⓘ and shown in Figure 11)	uOps sitting in an Issue Queue wait until all of their operands are ready and are then issued. This is the beginning of the out–of–order piece of the pipeline.	uOps are sent from the Frontend to the Backend, which includes the scheduler and reorder buffer (ROB). During this process, registers are renamed, and uOps are allocated to execution ports.	Picking from the scheduler to send a micro-op into an execution pipeline. Some macro ops may be issued as multiple micro-ops into different execution pipelines.



(a) RISC-V BOOM

(b) x86-64 Skyklake

Fig. 11: Processor microarchitecture comparison: RISC-V BOOM [16] (a) and x86-64 Skyklake [14] (b)

Authorized licensed use limited to: ShanghaiTech University. Downloaded on August 31,2026 at 04:37:43 UTC from IEEE Xplore. Restrictions apply.